



VÉRIFICATION STATIQUE DES SÉQUENCES D'APPELS AUX FONCTIONS COLLECTIVES MPI

LORENZO LUCAS -- ROBLOT
ALICIA PLATH

VENDREDI 21 NOVEMBRE 2025

ensile

2025-2026
PROJET DE COMPILATION AVANCÉE



Table des matières

1	Présentation du projet	2
2	Passe de compilation pour trouver les appels aux fonctions collectives MPI	3
2.1	Enregistrement de la passe	3
2.2	Enregistrement des appels aux fonctions collectives MPI	3
2.3	Représentation d'un CFG	5
3	Définition d'une numérotation permettant de suivre la position d'une fonction collective MPI dans la séquence d'appel	6
4	Utilisation de la notion de frontière de post dominance itérée pour retrouver le nœud d'origine du deadlock potentiel	9
4.1	Post-dominance	9
4.2	Frontière de post-dominance	9
4.3	Frontière de post-dominance itérée	10
4.4	Les ensembles	11
5	Affichage d'un warning à l'utilisateur	15
6	Gestion des directives	17
A	Références	19

1 Présentation du projet

L’objectif de ce projet est de réaliser un **plugin** [2] pour le compilateur **GCC** dont le but sera d’analyser les positions des collectives MPI [3] dans un code écrit en **C** afin que celui-ci ne réalise pas de deadlock.

MPI (Message Passing Interface) définit une norme avec des sous-routines, fonctions, objets et autres éléments pour développer des programmes parallèles dans un environnement à mémoire distribuée. MPI est implémentée dans plusieurs bibliothèques, notamment Open MPI et MPICH. [1] En effet, elle a contribué à l’essor du développement d’applications de communication de messages, que ce soit en **C**, **C++**, ou bien même **Fortran**.

La norme MPI exige que les appels aux fonctions collectives soient effectués dans le même ordre pour tous les processus MPI. Si cette séquence d’appels diffère entre plusieurs processus concernés par ces appels MPI, cela entraîne un deadlock, c’est-à-dire une situation où des processus restent bloqués dans un état d’attente, attendant que d’autres processus également bloqués répondent.

L’intérêt de ce projet est alors de réaliser une analyse à la compilation des fichiers pour vérifier si le positionnement des appels aux fonctions collectives MPI dans le code ne risque pas de générer un tel deadlock. Pour cela, il est nécessaire, pour chaque fonction, d’examiner tous les chemins du CFG (graphe de flot de contrôle créé par **GCC** permettant de représenter le flux d’exécution d’un programme) allant de la source (point d’entrée de la fonction) aux différents puits (points de sorties de la fonction). Pour chaque chemin existant entre la source et les puits, si la séquence d’appel est la même, alors tous les processus invoqueront les fonctions collectives MPI dans le même ordre. Au contraire, s’il existe des chemins avec des séquences d’appels différentes, il y a une possibilité pour que plusieurs processus prennent des chemins différents, n’invoquant pas les fonctions collectives dans le même ordre, et créant ainsi un deadlock. Dans ce cas, nous nous proposons d’avertir l’utilisateur avec un warning, et d’instrumenter le code pour vérifier la présence du problème et, le cas échéant, arrêter proprement le programme sans laisser le deadlock apparaître. Afin de limiter l’analyse dans le cas de programmes complexes, nous nous limiterons à l’analyse intra procédurale (fonction par fonction, sans considérer les interactions entre les fonctions). De plus, nous définissons des directives (**#pragma** en **C/C++**) pour spécifier, à la compilation, les fonctions à considérer pour notre analyse.

2 Passe de compilation pour trouver les appels aux fonctions collectives MPI

Dans un premier temps, nous allons réaliser une passe de compilation que nous allons introduire au bon endroit dans le compilateur GCC afin de pouvoir trouver les appels aux fonctions collectives MPI.

2.1 Enregistrement de la passe

Pour ce faire, nous développons alors un **plugin** (fichier `libplugin.so`). Il s'agit d'un bout de code chargé par le compilateur au moment de la compilation d'un fichier sous forme de bibliothèque dynamique. Un plugin doit contenir au minimum une fonction d'initialisation appelée `plugin_init`, ainsi que la déclaration d'une variable globale pré-définie (`plugin_is_GPL_compatible`) nécessaire pour déclarer la compatibilité du plugin à la licence GPL. Ce prototype est défini dans le header `gcc-plugin.h`.

Ensuite, nous définissons notre passe. La structure d'une passe est définie dans le header `tree-pass.h`. On introduit alors plusieurs autres éléments pour définir proprement notre passe :

- une structure (`pass_data`) contenant des informations génériques sur notre passe,
- une classe (`my_pass`) qui hérite de la classe `gimple_opt_pass` dans laquelle on définit plusieurs fonctions qui sont appelées au moment de l'exécution du plugin :
 - `clone` – Création d'une nouvelle instance d'une passe,
 - `gate` – Fonction de décision d'exécution,
 - `execute` – Fonction d'exécution de la passe.

La passe s'exécute quand la fonction `gate` renvoie le booléen `true`.

Une fois notre passe définie, nous l'insérons dans le processus de compilation. Pour cela, nous renseignons dans une structure de type `register_pass_info` (`my_pass_info`) les informations sur la passe et son insertion. Dans notre cas, nous précisons vouloir insérer la passe après que GCC ait créé son CFG.

Enfin, nous appelons la fonction d'enregistrement `register_callback` pour enregistrer les événements déclencheurs du plugin renseignés dans `my_pass_info`.

Nous avons alors initialisé notre plugin.

2.2 Enregistrement des appels aux fonctions collectives MPI

Nous considérons que notre plugin peut traiter les appels aux fonctions suivantes :

 – Listing 1 : Fichier `MPI_collectives.def` – 

```
1 DEFMPICOLLECTIVES( MPI_INIT, "MPI_Init" )
2 DEFMPICOLLECTIVES( MPI_FINALIZE, "MPI_Finalize" )
3 DEFMPICOLLECTIVES( MPI_REDUCE, "MPI_Reduce" )
4 DEFMPICOLLECTIVES( MPI_ALL_REDUCE, "MPI_AllReduce" )
```

```
5 DEFMPICOLLECTIVES( MPI_BARRIER, "MPI_Barrier" )
```

Nous introduisons aussi les fichiers `mpi_collectives.h` et `mpi_collectives.cpp` permettant à notre plugin de savoir quels appels MPI le plugin devra considérer grâce au fichier `MPI_collectives.def`, ainsi que la fonction `get_mpi_call_code` qui renvoie l'appel MPI détecté pour un ensemble d'instructions (*statements*) donné en entrée de la fonction.

Pour rappel, un bloc de base (*basic block*) est une séquence d'instructions consécutives où le flot d'exécution (contrôle) entre au début et ne sort qu'à la fin de la séquence. On souhaite partitionner tous les basic blocks comportant plusieurs appels MPI. C'est ce que fait la fonction `splitBlock` présente dans le fichier `plugin.cpp`.

– Listing 2 : Fonction `splitBlock` –

```
1  /**
2   * @brief Splits blocks containing multiple collectives.
3   *
4   * @param fun Pointer to the function currently being analyzed.
5   */
6  void splitBlock(function* fun) {
7      basic_block bb;
8      FOR_ALL_BB_FN(bb, fun) {
9          gimple_stmt_iterator gsi;
10         bool collective_found = false;
11         for (gsi = gsi_start_bb(bb); !gsi_end_p(gsi); gsi_next(&gsi)) {
12             gimple *stmt = gsi_stmt(gsi);
13             mpi_collective_code collective = get_mpi_call_code(stmt);
14             if (collective != LAST_AND_UNUSED_MPI_COLLECTIVE_CODE) {
15                 if (collective_found) {
16                     gsi_prev(&gsi);
17                     gimple* stmt_prev = gsi_stmt(gsi);
18                     split_block(bb, stmt_prev);
19                 }
20                 else {
21                     collective_found = true;
22                 }
23             }
24         }
25     }
26 }
```

Dans cette fonction, la boucle `FOR_ALL_BB_FN` parcourt chaque basic block. On parcourt ensuite l'ensemble des statements qu'il contient. On identifie alors chaque statement et on vérifie s'il correspond à un appel MPI grâce à la fonction `get_mpi_call_code`. Pour un basic block donné, s'il s'agit d'un appel MPI, alors on sait à la première itération sur

les statements que ce basic block contient au moins un appel MPI. Si on trouve d'autres appels MPI dans ce même basic block, alors on appelle la fonction `split_block` définie dans `cfghooks.h` pour partitionner le basic block.

Après avoir partitionné les basic blocks pour que chaque basic block contienne exactement un appel MPI, on utilise la fonction `saveCollective` définie dans `plugin.cpp`.

– Listing 3 : Fonction `saveCollective` –

```

1  /**
2   * @brief Records MPI collective operations in each basic block.
3   *
4   * @param fun Pointer to the function currently being analyzed.
5   * @param collective_tab Array of ID for each MPI collective operations
6   */
7  void saveCollective(function* fun, int* collective_tab) {
8      basic_block bb;
9      FOR_ALL_BB_FN (bb, fun) {
10         gimple_stmt_iterator gsi;
11         int collective_found = false;
12         for (gsi = gsi_start_bb (bb); !gsi_end_p (gsi); gsi_next (&gsi)) {
13             gimple *stmt = gsi_stmt (gsi);
14             mpi_collective_code collective = get_mpi_call_code(stmt);
15             if (collective != LAST_AND_UNUSED_MPI_COLLECTIVE_CODE) {
16                 collective_tab[bb->index] = (int)collective;
17                 collective_found = true;
18                 break;
19             }
20         }
21         if (!collective_found) collective_tab[bb->index] =
22             ↪ LAST_AND_UNUSED_MPI_COLLECTIVE_CODE;
23     }
24 }
```

Cette fonction enregistre dans un tableau¹ l'identifiant des collectives associées à chaque basic block selon l'indice du basic block (donné par `bb->index`).

2.3 Représentation d'un CFG

Une fonctionnalité du plugin est aussi de générer des fichiers Graphviz. Nous avons implémenté cette dernière dans les fichiers `graphviz.h` et `graphviz.cpp`. Cela permet au développeur d'avoir une représentation visuelle du CFG associé au code susceptible de provoquer un deadlock.

¹Le tableau est déclaré dans la fonction `execute` de la classe `my_pass`.

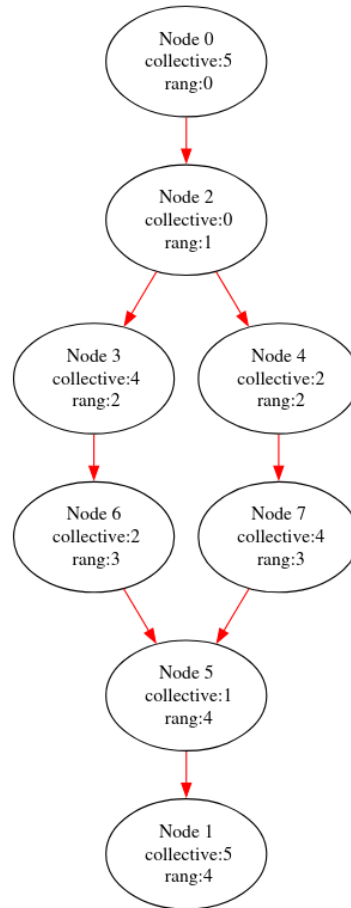
3 Définition d'une numérotation permettant de suivre la position d'une fonction collective MPI dans la séquence d'appel

Pour suivre la position d'une fonction collective MPI dans la séquence d'appel, nous avons utilisé la notion de rang. Le rang d'un nœud correspond au nombre maximal de collectives MPI rencontrées depuis l'entrée du programme jusqu'à ce nœud inclus, en ignorant les boucles.

La notion de rang joue un rôle essentiel dans la détection de deadlocks liés aux collectives MPI, car elle permet d'identifier la position logique d'une collective dans la séquence d'exécution d'un processus. Deux processus peuvent se bloquer si l'un attend une collective C1 tandis que l'autre attend une collective C2.

– Listing 4 : Fichier test_simple4.c –

```
1  int main(int argc, char *argv[]) {
2      MPI_Init(&argc, &argv);
3
4      int rank, size;
5      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
6      MPI_Comm_size(MPI_COMM_WORLD, &size);
7
8      int value = rank + 1;
9      int sum = 0;
10
11     if (rank == 0) {
12         MPI_Barrier(MPI_COMM_WORLD);
13         MPI_Reduce(&value, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
14     }
15     else {
16         MPI_Reduce(&value, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
17         MPI_Barrier(MPI_COMM_WORLD);
18     }
19
20     MPI_Finalize();
21     return 0;
22 }
```

Figure 1 : CFG associé au code `test_simple4`

Dans cet exemple, la collective 4 (`MPI_Barrier`) est bien présente sur tous les chemins, donc chaque processus devra l'exécuter. Cependant, l'ordre des collectives varie selon le chemin suivi. En effet, la collective 2 (`MPI_Reduce`) intervient avant la collective 4 sur une branche et après sur une autre. Les deux collectives ont donc des rangs différents selon le processus, ce qui conduit à un deadlock. Le numéro de la collective **et** son rang sont donc deux informations essentielles pour identifier un deadlock.

Enfin, dans le calcul du rang, il est non seulement possible mais nécessaire d'ignorer les boucles. Une boucle peut répéter plusieurs fois un segment de code, mais elle ne modifie jamais l'ordre relatif des collectives : une collective placée dans une boucle n'a pas d'autre position logique que « à cet endroit du programme ». Prendre en compte les répétitions introduirait des cycles qui feraient croître le rang à l'infini, sans ajouter la moindre information utile. En ignorant les boucles, on obtient un rang qui reflète uniquement l'ordre logique des collectives, ce qui est exactement ce qu'il faut pour raisonner correctement sur les conditions de synchronisation et donc sur les risques de deadlock.

Nous avons donc implémenté les mécanismes permettant d'éliminer les boucles et de calculer le rang de manière récursive. Pour le calcul du rang, le principe est simple : on ajoute 1 si le nœud courant contient une collective MPI, puis on calcule le rang de ses successeurs. Lorsqu'un nœud possède plusieurs prédécesseurs, son rang est défini comme

le maximum des rangs de ces prédécesseurs, car il correspond au plus grand nombre de collectives MPI rencontrées sur un chemin menant à ce nœud.

Pour l'élimination des boucles, chaque chemin depuis l'entrée du programme est exploré en maintenant une liste des nœuds déjà visités sur ce chemin. À chaque étape, un nœud vérifie qu'il n'apparaît pas déjà dans cette liste. S'il est présent, cela signifie qu'une boucle a été détectée et qu'il faut supprimer l'arête précédente. Sinon, il s'ajoute à la liste avant de la transmettre à ses successeurs.

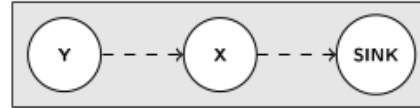
4 Utilisation de la notion de frontière de post dominance itérée pour retrouver le nœud d'origine du deadlock potentiel

Pour identifier un deadlock, nous allons utiliser le principe de **frontière de post-dominance itérée**. Pour comprendre son intérêt, nous allons d'abord définir les notions de post-dominance et de frontière de post-dominance.

4.1 Post-dominance

Un nœud X post-domine un nœud Y si X apparaît sur tous les chemins de Y vers le puits (SINK). Nous écrivons alors $X \gg_p Y$. L'ensemble des nœuds qui post-dominent Y est noté $\text{PDom}(Y)$. Un nœud X post-domine strictement un nœud Y si $X \neq Y$.

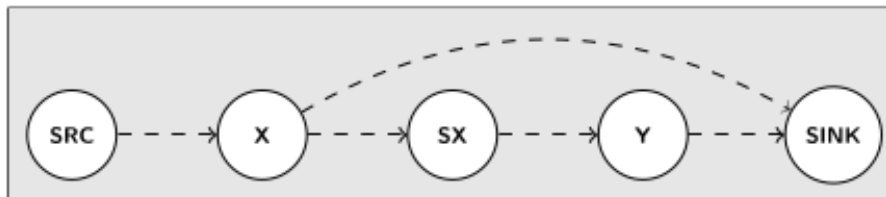
$$\text{PDom}(Y) = \left(\bigcap_{p \in \text{succs}(Y)} \text{PDom}(p) \right)$$



4.2 Frontière de post-dominance

La frontière de post-dominance d'un nœud y , notée $\text{PDF}(y)$, est l'ensemble des nœuds x tels que y post-domine un successeur de x , mais ne post-domine pas strictement x . Cela signifie que x est le premier nœud qui se trouve à la jonction du chemin allant de la source vers y et du chemin allant de la source (SRC) vers le puit (SINK) qui ne passe pas par y .

$$\text{PDF}(y) = \{x \mid \exists Sx \in \text{succs}(x), \text{ tel que } y \gg_p Sx \text{ et } y \not\gg_p x\}$$



La frontière de post-dominance correspond à la source potentielle du deadlock.

– Listing 5 : Fichier test_simple.c –

```

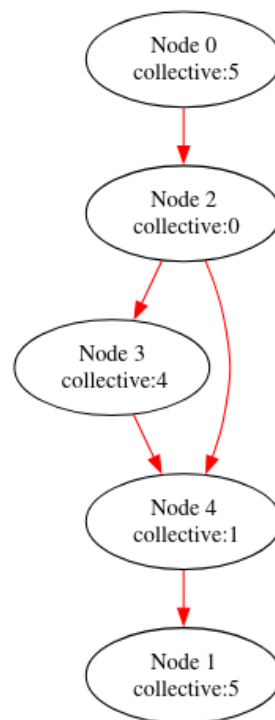
1  int main(int argc, char * argv[]) {           // -/
2      MPI_Init(&argc, &argv);                   // /
3                                              // /
4      int rank;                                // /
5      MPI_Comm_rank(MPI_COMM_WORLD, &rank);    // /--> Node 2

```

```

6                                     // /
7   if (rank == 0) {                 // /
8       MPI_Barrier(MPI_COMM_WORLD); // |--> Node 3
9   }                                 // /
10                                    // /
11   MPI_Finalize();                  // /
12   return 1;                        // |--> Node 4
13 }                                  // -/

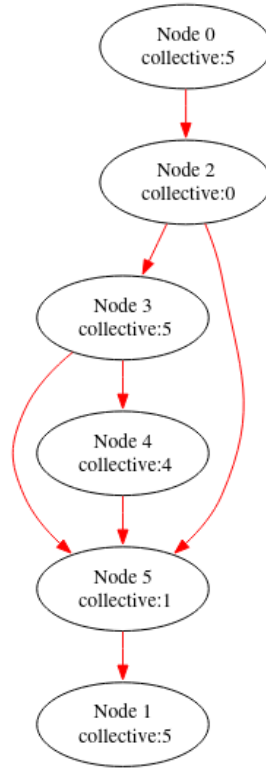
```

Figure 2 : CFG associé au code `test_simple`

Dans cet exemple, les collectives MPI sont présentes dans les nœuds 2, 3 et 4. La collective intéressante est celle du nœud 3. La frontière de post-dominance de ce nœud est le nœud 2 : c'est le premier nœud précédant 3 ayant un chemin vers la sortie qui ne passe pas par 3. Ainsi, la collective MPI située au nœud 3 peut créer un deadlock, car certains processus peuvent l'exécuter et d'autres non, et le nœud 2 en est la cause.

4.3 Frontière de post-dominance itérée

La frontière de post-dominance itérée correspond à l'union de toutes les frontières de post-dominance obtenues à chaque itération.



Considérons le nœud 4. Sa frontière de post-dominance est 3. La frontière de post-dominance du nœud 3 est 2. Enfin, le nœud 2 n’a pas de frontière de post-dominance. Ainsi, la frontière de post-dominance itérée du nœud 4 est $\{2, 3\}$. Cela correspond aux intersections qui permettent à un processus de ne pas passer par le bloc 4. Étant donné que le bloc 4 est le seul de son rang à posséder cette collective MPI, un deadlock potentiel peut survenir à cause des blocs 2 et 3.

4.4 Les ensembles

La section 3 nous a permis de mettre en place les rangs et de comprendre leur importance. On ne peut donc pas s’intéresser à un bloc isolé, mais à l’ensemble des blocs ayant le même rang et la même collective MPI. Le principe reste le même : pour chaque bloc, il faut vérifier que sa frontière de post-dominance itérée est vide. Si ce n’est pas le cas, cela signifie qu’un chemin existe permettant à un processus de ne pas exécuter la collective MPI, et donc qu’un deadlock est possible.

Pour calculer la frontière de post-dominance d’un ensemble, nous utilisons la formule suivante :

$$\text{PDF}(E) = \{X \mid \exists S \in \text{succs}(X), \text{ tel que } E \gg_p S \text{ et } E \not\gg_p X\}$$

avec la post-dominance définit par :

$$E \gg_p X \Leftrightarrow \forall S \in \text{succs}(X), \exists Y \subset E, \text{ tel que } Y \gg_p S$$

Nous commençons par la fonction `doesEPostDomineX`, qui, comme son nom l’indique,

permet de vérifier si un ensemble E post-domine un noeud X . Cette fonction est directement basée sur la formule précédente. De plus, si un sous-ensemble Y de E post-domine un noeud S alors E post-domine S . La fonction a été implémentée de la manière suivante :

– Listing 6 : Fonction doesEPostDomineX –

```

1  /**
2   * @brief Checks if a node (basic block) @p x post-dominates a set @p e.
3   *
4   * @param fun Pointer to the function currently being analyzed.
5   * @param e Bitmap representing the initial set of basic blocks (E).
6   * @param x Basic block
7   * @return bool Return true if the node (basic block) @p x post-dominates
8   *         ↪ the set @p e, false otherwise.
9   */
9  bool doesEPostDomineX(function* fun, bitmap_head* e, basic_block x,
10 ↪ bitmap_head visited) {
11      if (bitmap_bit_p(e, x->index)) return true;
12
13      if (EDGE_COUNT(x->succs) == 0) return false;
14
15      if (bitmap_bit_p(&visited, x->index)) return true;
16
17      edge p;
18      edge_iterator ei;
19
20      FOR_EACH_EDGE(p, ei, x->succs) {
21          bitmap_set_bit(&visited, x->index);
22          if (!doesEPostDomineX(fun, e, p->dest, visited)) return false;
23      }
24      return true;
25  }

```

Cette fonction comporte une exception importante : un ensemble E post-domine toujours un noeud X s'il appartient à E . Dans ce cas, l'équivalence de la formule générale n'est plus valable, ce qui est géré par la première condition `if`.

Nous sommes assurés que la fonction termine toujours : si E post-domine X , nous atteindrons un ou plusieurs noeuds appartenant à E . Sinon, nous atteindrons la fin du graphe.

Les boucles sont également prises en compte. Si une boucle est parcourue sans rencontrer un élément de E , la fonction retourne `true`. Cela est logique, car cette boucle n'a pas d'impact réel sur la post-dominance : le programme finira par sortir de la boucle et explorer un autre chemin que nous explorons déjà, étant donné que tous les successeurs sont visités à chaque appel récursif.

Pour calculer la frontière de post-dominance, il suffit ensuite d'appliquer la formule précédente à l'aide de la fonction `doesEPostDomineX`. La fonction suivante ne présente

pas de difficultés particulières : les successeurs d'un nœud S ne sont parcourus que si S n'est pas post-dominé par l'ensemble E .

– Listing 7 : Fonction `frontierePostDominance` –

```

1  /**
2   * @brief computes the post-dominance frontier of a set
3   *
4   * @param fun Pointer to the function currently being analyzed.
5   * @param e Bitmap representing the initial set of basic blocks (E).
6   * @param pdf Bitmap representing the resulting iterated post-dominance
7   *   ↪ frontier.
8   * @return bool TODO
9   */
10 bool frontierePostDominance(function* fun, bitmap_head* e, bitmap_head*
11   ↪ pdf) {
12     bool doesPdfEmpty = true;
13
14     basic_block bb;
15     FOR_ALL_BB_FN(bb, fun) {
16         if (doesEPostDomineX(fun, e, bb)) continue;
17
18         edge p;
19         edge_iterator ei;
20
21         FOR_EACH_EDGE (p, ei, bb->succs) {
22             if (doesEPostDomineX(fun, e, p->dest)) {
23                 bitmap_set_bit(pdf, bb->index);
24                 doesPdfEmpty = false;
25             }
26         }
27     }
28     return doesPdfEmpty;
29 }

```

Enfin, pour la frontière de post-dominance itérée, nous utilisons des opérations sur les bitmaps afin de rechercher efficacement les nœuds concernés et de détecter d'éventuelles boucles. La fonction continue d'appliquer la frontière de post-dominance jusqu'à ce que celle-ci soit vide ou que les nouveaux éléments calculés soient déjà présents dans la frontière itérée.

– Listing 8 : Fonction `frontierePostDominanceIteree` –

```

1  /**
2   * @brief Computes the iterated post-dominance frontier of a given set of
3   *   ↪ basic blocks.
4   *

```

```

4      * @param fun Pointer to the function currently being analyzed.
5      * @param pdf Bitmap representing the resulting post-dominance frontier.
6      * @param pdf_iter Bitmap representing the memory space to store the
    ↪ resulting iterated post-dominance frontier
7      */
8      void frontierePostDominanceIteree(function* fun, bitmap_head* pdf,
    ↪ bitmap_head* pdf_iter)
9      {
10         bool isPdfEmpty = false;
11         while (!isPdfEmpty && !bitmap_intersect_p((const bitmap)pdf_iter,
    ↪ (const bitmap)pdf)) {
12             bitmap_head copy;
13             bitmap_initialize(&copy, &bitmap_default_obstack);
14             bitmap_copy(&copy, pdf_iter);
15             bitmap_ior(pdf_iter, (const_bitmap)&copy, (const_bitmap)pdf);
16
17             bitmap_head pdf_of_pdf ;
18             bitmap_initialize (&pdf_of_pdf, &bitmap_default_obstack);
19             isPdfEmpty = frontierePostDominance(fun, pdf, &pdf_of_pdf);
20
21             if(!isPdfEmpty) bitmap_copy(pdf, &pdf_of_pdf);
22         }
23     }

```

Dans ce code, les méthodes des bitmaps permettent de vérifier que nous ne sommes pas dans une boucle infinie. La recherche de la frontière itérée se poursuit tant que celle-ci n'est pas vide et que les nouveaux éléments ne sont pas déjà inclus dans la frontière existante.

Ces fonctions permettent de déterminer précisément quels nœuds ou ensembles de nœuds peuvent poser problème pour les collectives MPI. La fonction `doesEPostDomineX` identifie la post-dominance, la fonction `frontierePostDominance` calcule la frontière, et la fonction `frontierePostDominanceIteree` étend ce calcul de manière itérée. Grâce à ce mécanisme, on peut détecter efficacement les sources potentielles de deadlocks dans un programme MPI.

5 Affichage d'un warning à l'utilisateur

Dans les parties précédentes, nous avons identifié les ensembles de collectives MPI susceptibles d'occasionner des deadlocks ainsi que leurs causes : les blocs à l'origine des branches divergentes. Il s'agit maintenant d'afficher un warning compréhensible destiné à informer l'utilisateur de ces risques potentiels.

Pour cela, nous utilisons la fonction de GCC `warning_at` qui permet de générer un avertissement à un emplacement précis. Nous avons choisi de considérer que les blocs de séparations (en plusieurs branches) identifiés sont plus responsables du potentiel deadlock que les collectives MPI elles-mêmes. Ainsi, c'est la ligne correspondant à cette séparation qui est signalée. Le message associé indique ensuite clairement quelles collectives MPI sont concernées.

>_

```
...
std::string eString = "this intersection causes some problems
on the MPI collective(s) line: ";
...
```

De plus, nous considérons que chaque séparation susceptible de provoquer un deadlock pour un ensemble donné de collectives MPI doit être signalée individuellement. Chaque jonction correspond en effet à un problème distinct apparaissant à un niveau différent du graphe.

Pour un ensemble de collective MPI E et sa frontière de post-dominance itérée PDF, nous concaténons dans le message d'erreur les numéros de lignes où apparaissent les collectives concernées. Ensuite, pour chaque élément de la frontière de post-dominance, c'est-à-dire pour chaque séparation problématique, nous affichons un warning indiquant quelles collectives MPI sont menacées à cause de cette séparation.

Enfin, pour un bloc de séparation, nous avons choisi de pointer vers le dernier statement du bloc. Il s'agit en pratique de la dernière ligne avant la séparation, souvent associée à un `if`, `for` ou `while`. Nous sommes conscients que le compilateur découpe ces instructions en plusieurs statements et que l'avertissement ne pourra pas pointer exactement toute la ligne utilisateur. Néanmoins, cette précision reste suffisante pour permettre à l'utilisateur d'identifier clairement le problème, d'autant que notre message précise explicitement qu'il s'agit d'une "intersection".

– Listing 9 : Fonction `messageErreur` –

```
1  /**
2   * @brief Emits a warning message for problematic MPI collective
3   *        ↪ intersections.
4   *
5   * @param fun Pointer to the function currently being analyzed.
6   * @param pdf Bitmap representing the resulting iterated post-dominance
7   *        ↪ frontier.
```



```

6      * @param e Bitmap representing the initial set of basic blocks (E).
7      */
8      void messageErreur(function* fun, bitmap_head* pdf, bitmap_head* e) {
9          std::string eString = "this intersection causes some problems on the
          ↳ MPI collective(s) line(s): ";
10
11         basic_block bb;
12         FOR_ALL_BB_FN(bb, fun) {
13             if (bitmap_bit_p(e, bb->index)) {
14                 gimple_stmt_iterator gsi;
15                 gsi = gsi_start_bb(bb);
16                 gimple *stmt = gsi_stmt(gsi);
17
18                 while (get_mpi_call_code(stmt) ==
19                     ↳ LAST_AND_UNUSED_MPI_COLLECTIVE_CODE) {
20                     gsi_next(&gsi);
21                     stmt = gsi_stmt(gsi);
22                 }
23                 eString += std::to_string(gimple_lineno(stmt)) + " ";
24             }
25
26         FOR_ALL_BB_FN(bb, pdf) {
27             if (bitmap_bit_p(pdf, bb->index)) {
28                 gimple_stmt_iterator gsi;
29                 gsi = gsi_start_bb(bb);
30                 while(!gsi_one_before_end_p(gsi)) gsi_next(&gsi);
31
32                 gimple *stmt = gsi_stmt (gsi);
33                 location_t loc = gimple_location(stmt);
34
35                 warning_at(loc, 0, "%s", eString.c_str());
36             }
37         }
38     }

```

L'utilisateur peut ainsi identifier rapidement l'origine du problème et le corriger.

6 Gestion des directives

Afin de sélectionner les fonctions sur lesquelles nous souhaitons appliquer notre vérification, nous utilisons le système de *pragmas* offert par GCC. Ainsi, pour activer la vérification sur une fonction ou plusieurs fonctions, il suffit d'écrire :

- `#pragma ProjetCA mpicoll_check mafunc`
- `#pragma ProjetCA mpicoll_check(mafunc1,mafunc2,mafunc3)`

Tout d'abord, nous devons déclarer cette nouvelle directive. L'enregistrement du pragma dans la phase d'initialisation du plugin permet de définir son nom ainsi que la fonction qui sera appelée lorsque la directive sera rencontrée.



```
c_register_pragma("ProjetCA", "mpicoll_check", my_pragma_handler);
```

Afin de gérer correctement la directive `#pragma`, nous vérifions d'abord qu'elle respecte bien la forme attendue : elle ne doit pas apparaître à l'intérieur d'une fonction, elle doit être correctement formée et respecter la syntaxe imposée. Si ce n'est pas le cas, une erreur est générée.



```
test_simple.c:13:13: erreur: « #pragma GCC option » n'est pas permis
↳ à l'intérieur des fonctions
  13 |      #pragma ProjetCA mpicoll\_check main
      |              ~~~~~
```

Nous parcourons ensuite les différents paramètres afin de déterminer quelles fonctions doivent être analysées. Un *warning* est émis lorsqu'un même nom de fonction est mentionné plusieurs fois dans une même directive.



```
test_simple.c:7:33: attention: The function name: main appears more
↳ than 1 time
   7 | #pragma ProjetCA mpicoll_check (main, main)
      |                               ~~~~
```

Les fonctions à analyser sont ensuite enregistrées dans une variable globale. La fonction `gate` de la passe (qui décide si la passe doit s'appliquer sur une fonction donnée) se contente alors de vérifier si le nom de la fonction courante apparaît dans cette liste. Elle retourne `true` si la passe doit être exécutée sur cette fonction, `false` sinon.

Enfin, nous avons mis en place un *callback* final, exécuté lors de la fermeture de GCC. Celui-ci vérifie que l'ensemble des fonctions enregistrées dans les pragmas ont bien été rencontrées au cours de la compilation. Si certaines fonctions n'ont jamais été définies,

un *warning* est affiché : cela signale probablement une fonction déclarée dans le *pragma* mais absente du code.



```
test_simple.c:23:1: attention: function 'chat' defined in pragma
↳ instruction but not used
  23 | }
     | ^
```

Enfin, nous avons décidé que l'utilisateur pouvait également ne pas définir de *pragma*. Dans ce cas, toutes les fonctions sont analysées. Pour cela, nous avons créé une variable globale, un booléen indiquant si une directive **pragma** est présente ou non.

Cette partie nous a permis d'introduire un *pragma* personnalisé pour sélectionner les fonctions à analyser. Nous avons vérifié la validité de la directive, enregistré les fonctions demandées, et évité les doublons. La passe du plugin ne s'active ensuite que pour ces fonctions. Enfin, un contrôle final signale les fonctions mentionnées dans un *pragma* mais jamais rencontrées. Cela pose les bases nécessaires pour appliquer nos vérifications uniquement là où l'utilisateur le souhaite.

A Références

- [1] Alliance. MPI. <https://docs.alliancecan.ca/wiki/MPI/fr>.
- [2] Free Software Foundation. Plugins – GNU Compiler Collection (GCC) Internals. <https://gcc.gnu.org/onlinedocs/gcc-12.2.0/gccint/Plugins.html#Plugins>.
- [3] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard Version 5.0*, June 2025.